# Lab 28/1

→ Viết chi tiết và demo thực hành đi kèm(chưa có)

Note lại mình đã học gì:

- Tìm hiểu machine learning workflow mlops
- các dạng bài toán chính
- tìm hiểu về tiền xử lý dữ liệu như nào

- mô hình transformer hiểu nguyên lý phần transformer

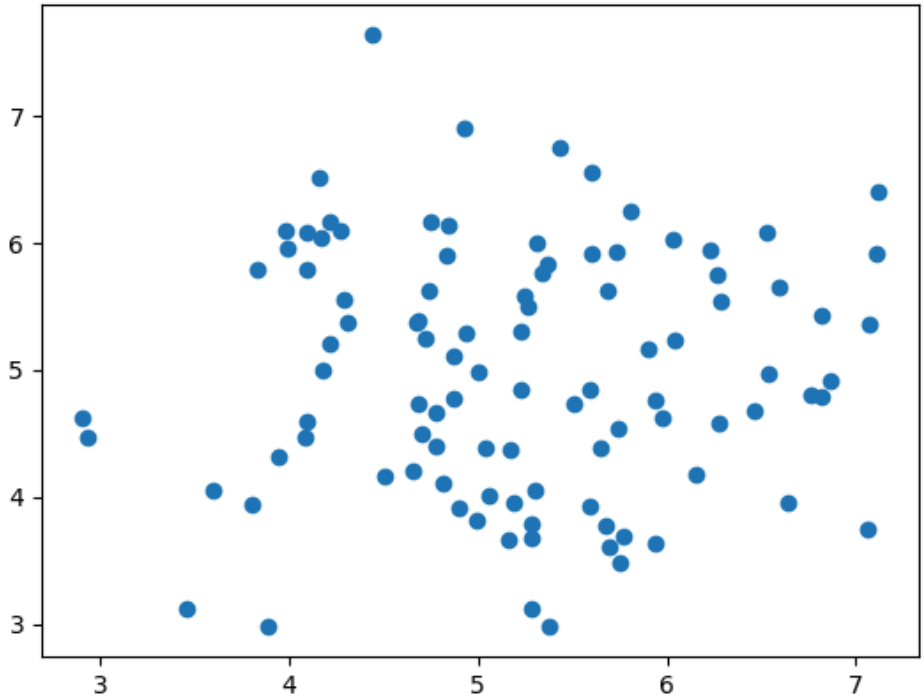
## Tiền xử lý dữ liệu

## Data Cleaning

- Tiền xử lý dữ liệu nhắc tới việc chuẩn bị và biến đổi các dư liệu thô thành các dữ liệu mà cách model học máy có thể sử dụng được. Việc mình preprocess các dữ liệu ấy sẽ giúp cải thiện chất lượng cũng như là độ hiệu quả của ML.
- Việc tiền xử lý này bao gồm:
  - Xử lý các giá trị còn thiếu
    - Để tìm ra các giá trị còn thiếu thì mình có thể dùng các công cụ phác hoạ hoặc thống kê để tìm ra các cột/feature đang thiếu giá trị.
    - Nếu mà tỷ lệ thiếu thấp và không quan trọng thì có thể xoá hoàn toàn bằng các hàm trong các thư viện.
    - Với các giá trị số học thì mình có thể điền bù vào bằng các giá trị trung bình, chính giữa,.... Nếu mà là trong bài toán phân loại thì có thể điền vào category phổ biến nhất
  - Xử lý các giá trị ngoại lệ:
    - Ví dụ: các giá trị ngoại lệ, chẳng hạn 1 căn nhà biệt thự 2000 mét vuông giá 200 tỷ trong khi nhà bình thường khoảng 100m2, sẽ khiến

linear regression bị nghiêng về điểm đấy → prediction bị lệch

- Có thể tìm ra các giá trị này bằng cách minh hoạ data của mình thành một scatterplot




- Trong một số trường hợp thì mình có thể loại bỏ hoàn thoàn các điểm như vậy. Hoặc có thể xử dụng các kỹ năng lọc như một ngưỡng xác suất để loại bỏ chúng.

- Xử lý các giá trị lặp
  - Các giá trị lặp có thể thể dẫn tới việc một số điểm, hoặc bias bị nhấn mạnh
  - Có thể dùng các hàm của các thư viện như duplicated() trong pandas để tìm ra.
  - Nếu thông tin lặp lại ấy không quan trọng thì có thể loại bỏ bằng drop_duplicates() trong pandas hoặc các hàm tương tự khác trong thư viện.

## Data Normalization

Mục đích của data Normalization là đưa tất cả các feature lại thành một khoảng nhất định mà không làm biến đổi sự khác biệt giữa các khoảng của chúng. Cái này là để khi chạy học máy sẽ hiệu quả và converge nhanh hơn.

→ Giảm thiểu ảnh hưởng của các giá trị ngoài lề.

Một số phương thức normalization:

1.Min - max scaling:

- Formula:Xnorm=X−Xmin/Xmax−Xmin
- Range: Khoảng giá trị rơi vào giữa 0 và 1.

→ Phù hợp cho các thuật toán đòi hỏi input feature nằm trong một khoảng nhất định, chẳng hạn neural network. Lưu ý rằng cái này dễ bị ảnh hưởng bởi outliers.

2.Z-score normalization

- Formula:

$$x_{new} = \frac{x - \mu}{\sigma}$$

- Range: Khoảng giá trị có thể thay đổi nhưng sẽ giao động quanh 0.
- Transforms values to have a mean of 0 and standard deviation of 1.

→ Phù hợp cho các thuật toán như kiểu k-means cluttering, linear regression, logistic regression

## Feature Scaling

- Scaling Feature giúp chuẩn hoá giá trị của các variable độc lập hoặc feature của data set. Feature Scaling giúp đưa tất cả các feature về một khoảng cụ thể, phòng cho trường hợp một feature lớn sẽ đàn áp các feature khác nhỏ hơn trong quá trình học máy.
- Feature Scaling giống Data Normalization là một phần của feature scaling

- Ngoài 2 cách trong Data Normalization mình có:
  - MaxAbsScaler: Chia cho giá trị tuyệt đối lớn nhất của Feature đó:

$$x_{new} = \frac{x}{|x|_{max}}$$

→ Phạm vi sẽ nằm trong [-1;1] hoặc [0;1] nếu dữ liệu gốc toàn dương.

→ Phù hợp cho các data set chứa các dữ liệu thưa, khi mà việc mình đảm bảo giữ nguyên các giá trị là "0" quan trọng, chẳng hạn trong phân tích ký tự và chữ cái.

  - RobustScaler (tốt cho khi có nhiều outliers): Sử dụng Median và IQR Mean và Standard Deviation:
    - Median là giá trị đứng giữa nên không bị ảnh hưởng bởi giá trị ngoại lai.
    - IQR chỉ quan tâm tới 50% số người ở giữa(25% → 75%).

$$x_{new} = \frac{x - Median}{IQR}$$

## Handling Catergorical Data

- Non-numeric: Những Data dạng này được đại diện bằng chuỗi ký tự hoặc nhãn, và các thông tin như này không thể được xử lý tực tiếp.
- Ordinal and Nominal Variables: Các catergorical data có thể ordinial (có thứ tự) hoặc nominal(không có thứ tự cụ thể). Nhầm lần giữa 2 loại này có thể dẫn tới sai xót.

  →Nominal: Chó, Mèo, Ngựa,...

  →Ordinal: Cấp 3, Đại Học, Thạc Sĩ, Tiến Sĩ
- Các cách để encode các biến dạng loại:
  - Label Encoding: Gán một ký tự cho mỗi nhóm cố định. Phù hợp cho các biến theo thứ tự, có một thứ tự có ý nghĩa giữa các nhóm khác nhau

→ LabelEncoder() trong python's scikit-learn

- One-hot encoding: Tạo ra các cột chỉ trứa giá trị nhị phân, biểu thị sự có hoặc không có mặt của giá trị ấy. (vd màu sắc thì mình có 3 cột đỏ xanh vàng). Tốt cho giá trị nominal.
- Dummy Encoding: Giống với One-Hot Encoding nhưng bỏ đi một cột, chẳng hạn nếu một màu có 3 cột đỏ, vàng, xanh thì ta sẽ bỏ cột đỏ đi, nếu cột vàng và xanh đều mang giá trị 0, thì có thể suy ra đó là màu đỏ.

  → Thường sử dụng trong các mô hình regression, khi một nhóm đóng vai trò làm nhóm đối chiếu.

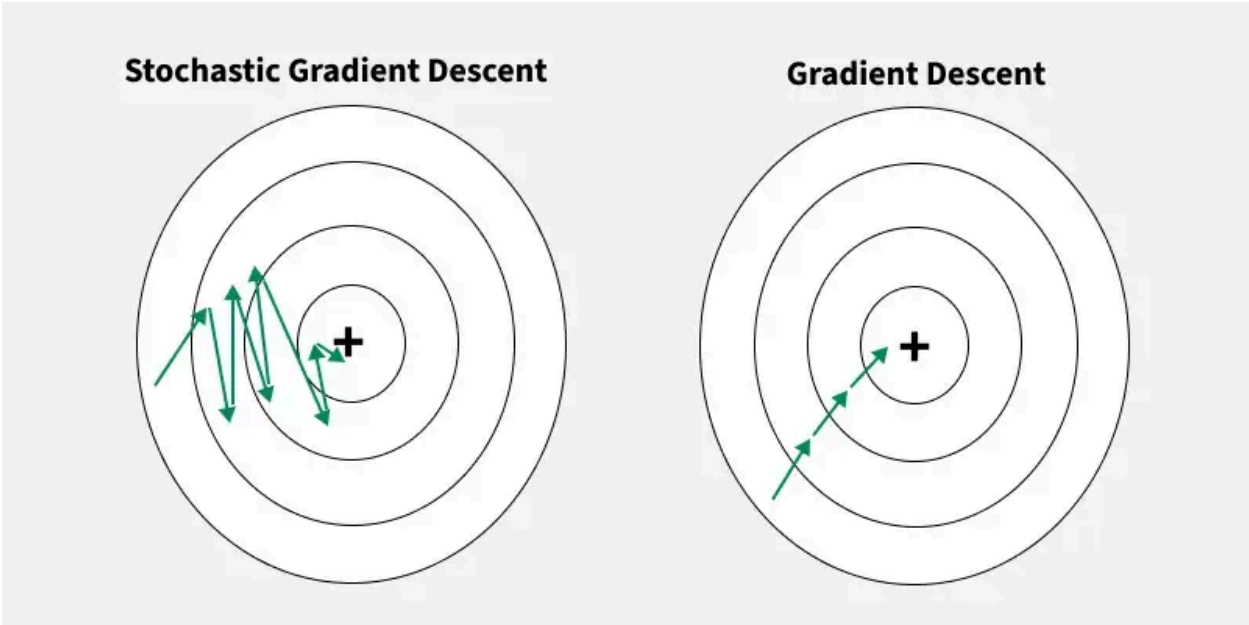
## Các Dạng Bài Toán Chính

Để hiểu được các thuật toán (Optimization Algorithms) này, em nghĩ sẽ chia thành nhiều bậc khác nhau. Với bậc, là bậc của đạo hàm được sử dụng. Tới thời điểm hiện tại em mới học về Gradient Descent, bậc 1, nên em sẽ tìm hiểu về các thuật toán cùng bậc đó. Đây là 3 thuật toán thường được sử dụng mà em tìm được:

- SGD kèm Momentum(SGD with momentum)
- Adam (Adaptive Moment Estimation)
- RMSprop (Root Mean Square Propagation)

## SGD with Momentum

Stochastic Gradient Descent là biến tấu của Gradient Descent nhưng kèm theo một số ưu điểm về độ hiệu quả, và scalability.




- Trong Gradient Descent cổ điển, gradient được tính từ cả dataset, dẫn tới việc tính toán cho các dataset lớn khó khăn. Trong SGD, graident đươc tính từ mỗi training example, thay vì cả dataset.

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta J(\theta_t; x^{(i)}, y^{(i)})$$

$$\theta_{t+1} = \underbrace{\theta_t}_{\text{Trọng số cũ}} - \underbrace{\eta}_{\text{Learning Rate}} \cdot \underbrace{\nabla J(\dots)}_{\text{Gradient}}$$

- $\theta_{t+1}$: Trọng số **mới** (sau khi cập nhật).
- $\theta_t$: Trọng số **hiện tại**.
- $\eta$ (eta): **Learning Rate** (Tốc độ học, ví dụ: 0.01). Đây là độ dài bước đi.
- $\nabla_\theta J$: **Gradient** (Đạo hàm riêng) của hàm mất mát.
- $(x^{(i)}, y^{(i)})$: **Một mẫu dữ liệu đơn lẻ** được chọn ngẫu nhiên từ tập dữ liệu (đây là điểm khác biệt chính so với Gradient Descent thông thường dùng toàn bộ dữ liệu).
- $-$: Dấu trừ thể hiện việc đi **ngược chiều** đạo hàm để xuống dốc (giảm lỗi).

- Điểm khác biệt quan trọng là việc cập nhật tham số giờ được tính từ một điểm data duy nhất

→ Nhược điểm của SGD cổ điển là việc converge bị nhiễu và giao động mạnh, do nó chỉ dùng một mẫu dữ liệu (hoặc một batch nhỏ). Dễ bị kẹt tại cực tiểu cục bộ, và nhạy cảm với việc điều chỉnh learning rate.

- Tại sao cần momentum:
- Nhận thấy rằng SGD cổ điển luôn đi sáng trái và phải liên tục (oscillation) mà không đi xuống thẳng, khi mình kèm theo momentum, nó sẽ ghi nhớ bước đi trước và cộng gộp các bước nhảy trái phải lại với nhau để triệt tiêu sự dao động.
- Dần dần khi bước đi thẳng thì nó sẽ cộng dồn tiếp tục theo hướng đi đó.

$$v_{t+1} = \gamma \cdot v_t + \eta \cdot \nabla_\theta J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

→ Vận tốc mới bằng momentum nhân với vận tốc cũ, cộng với learning rate nhân với đạo hàm của cost function (độ dốc của J tại vị trí theta t)

→ Sau đó mình update theta (weight và bias mà mình muốn tối ưu), bằng theta cũ trừ đi vận tốc mới

Chú thích:

gamma là ký hiệu momentum (thường là 0.9), kiểu muốn quyết định xem mình muốn giữ bao nhiêu % đà của quá khứ.

$$v_{t+1} = \underbrace{\gamma \cdot v_t}_{\text{Đà từ quá khứ}} + \underbrace{\eta \cdot \nabla_\theta J(\theta_t)}_{\text{Gia tốc hiện tại}}$$

Cập nhật vị trí:

$$\theta_{t+1} = \theta_t - \underbrace{v_{t+1}}_{\text{Vận tốc gộp}}$$

- Sử dụng SGD with Momentum giúp tránh cực tiểu cục bộ, giảm giao động, và converge nhanh hơn.

## RMSProp

RMSProp là một cải tiến của Gradient Descent và tập trung duy nhất vào việc tự động điều chỉnh tốc độ học dựa trên độ dốc của địa hình. (Làm sao để không đi quá nhanh ở chỗ dốc đứng và không đi quá chậm ở chỗ bằng phẳng)

Ta tính gradient với tham số cũ:

$$g_t = \nabla_\theta f(\theta_{t-1})$$

Tính độ dốc đã được tích luỹ từ lịch sử độ lớn của các con dốc đã đi qua:

$$v_t = \beta v_{t-1} + (1-\beta) g_t^2$$

- B: Decay Rate: thường đặt là 0.9 hoặc 0.99, nhắc tới việc nên nhớ bao nhiều về độ dốc trong quá khứ?
- v_t lớn → địa hình dốc gắt, thay đổi mạnh
- v_t nhỏ → địa hình phẳng, êm ả
- g_t^2 : bình phương đạo hàm của cost function để chỉ quan tâm tới độ lớn và loại bỏ phần âm.

Cập nhật bước nhảy:

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{v_t} + \epsilon} \cdot g_t$$

Nhìn vào công thức ta thấy nếu mà gt^2 lớn → vt lớn → mẫu số căn vt lớn → phân số a/căn vt nhỏ → bước nhảy bị thu lại

→ Tương tự cho điều ngược lại

- RMSProp sẽ được sử dụng ở thuật toán Adam ở dưới. Mình đã viết phần adam trước khi viết RMSProp nên phần này viết không kỹ lắm.

## Adam

- Viết tắt cho Adpative Movement Estimation. Adam là sự kết hợp ưu điểm của 2 thuật toán phổ biến: Momentum(giữ đà) và RMSprop(tự động điều chỉnh Learning Rate cho từng tham số riêng biệt - Adaptive Learning Rate)

Các bước:

1.Tính Gradient tại thời điểm t:

$$g_t = \nabla_\theta f(\theta_{t-1})$$

- "Tại bước hiện tại (t), hãy nhìn vào vị trí cũ (t-1) mà chúng ta đang đứng (theta). Sau đó, tính toán độ dốc (nabla) của ngọn núi (f) tại vị trí đó để xem hướng nào dốc nhất. Kết quả độ dốc đó gọi là g_t."

2.Cập nhật momen bậc 1: (tính đà di chuyển từ quá khứ)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

- momen mới (mt) được tính bằng cách lấy 90%(B1=0.9) của đà cũ m_t-1, cộng với 10% độ dốc mới vừa quan sát ((1-B1).gt)
- B1 là hệ số suy giảm, mình thường có thể chọn được là 0.9
- 1-B1 là phần còn lại, nói về độ quan trọng của thông tin mới(gradient thời điểm t là gt)

→ kiểu độ dốc chỉ có ảnh hưởng 10% vào hướng đi hiện tại

3.Momen bậc 2 - RMSprop

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- avt: second moment vector đại diện cho độ biến động trung bình của địa hình.
- B2: Hệ số suy giảm cho momen bậc 2 (thường là 0.999), do muốn ghi nhớ lịch sử độ biến động trong một khoảng thời gian rất dài.
- v_t-1: momen bậc 2 trong quá khứ
- g_t^2: bình phương của gradient, chỉ quan tâm tới độ lớn, không quan tâm tới hướng đi (do luôn dương)

→ tình toán độ gập ghềnh mới (vt) bằng cách giữ lại 99.9% thông tin về độ gập ghềnh trong quá khứ, cộng thêm một chút (0.01%) của độ lớn con dốc hiện tại.

- Mình làm như này là do trong bước 5, nếu vt lớn thì thuật toán sẽ chia nhỏ bước đi đi

4.Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Ban đầu mt và vt sẽ được khởi tạo bằng 0,  nên ở bước này ta sẽ chuẩn hoá điều đó.

→ Ta sẽ phóng to các giá trị m và v lên để bù đắp cho việc nó đc khởi tạo bằng 0 ở ban đầu, khi đã chạy đc lâu (t lớn0 thì hãy để nguyên chúng vì lúc này đã chuẩn xác

→ Mình sử dụng 1-B^t là do chẳng hạn t = 1 thì 1 - B^t = 0.1 → nhân với 10 để bù đắp cho việc ban đầu nó nhỏ. Sau khi lớn(t = 100, 0.9^100) thì mẫu số 1 - B^t sẽ tự động giữ nguyên giá trị (không cần sửa lỗi nữa)

5.Cập nhật tham số

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Tính được hướng đi và độ gập ghềnh được chuẩn hoá ta sẽ thay đổi các trọng số thsu

- m_t_hat: hướng đi đã sửa
- căn v_t_hat: căn bậc 2 của độ biến động, lấy từ bước 4 và khai căn (cái này tạo nên sự thích ứng của adam)

    - Nếu v lớn (có sự thay đổi mạnh) → mẫu số lớn → phân số nhỏ đi → bước nhảy thu nhỏ lại để an toàn
    - Tương tự điều ngược lại
- Epsilon là một số cực kỳ nhỏ (10^-8) thường để tránh đi việc chia cho 0)

1. **Tính độ dốc** (Gradient).
2. **Tính đà** (Momentum - $m$).
3. **Tính độ biến động** (RMSprop - $v$).
4. **Sửa lỗi khởi động** (Bias Correction - $\hat{m}, \hat{v}$).
5. **Cập nhật vị trí** (Update - $\theta$).

## Machine Learning Workflow

- ML workflow là một quá trình có hệ thống của việc phát triển, training, đánh giá và sử dụng các mô hình ML. Nó bao gồm một chuỗi các bước giúp cho những người thực hiện xuyên suốt quá trình của một dự án ML, từ việc định nghĩa vấn đề và áp dụng câu trả lời.

Các bước chính trong một ML Workflow:

1. Problem Definition
2. Data collection and preprocessing
3. Exploratory data analysis
4. Model selection and training
5. Model evaluation and tuning
6. Model deployment and monitoring

1. Problem Definition

- Formulate a problem statement: Tạo một câu hỏi rõ ràng và ngắn gọn. Câu hỏi này cần bao gồm những gì cần được thực hiện, phân loại và cải tiến. Nó cũng cần cân nhắc tới các yếu tố như là tihs khả thi, độ có sẵn của dữ liệu và các ảnh hưởng tiềm năng.
- Define Success criteria: Tạo ra các tiêu chí để đánh giá mức độ thành công, cũng như là các KPI mà có thể được sử dụng để đánh giá quá trình của ML. Nó phải đi liền với câu hỏi ở trên.
- Identify data requirements and constraints: Xác định yêu cầu về data, cũng như là các hạn chế. Bao gồm data types, nguồn thông tin, xem xét chất lượng, và các yếu tố đạo đức liên quan tới việc sử dụng thông tin.
- Risk assessment:  Xem xét rủi ro liên quan tới chất lượng data, độ phức tạp của model, regulatory compliance, ...

→ Viết lại tất cả các yếu tố trên vào một document, document này sẽ được sử dụng xuyên suốt quá trình ML để tham chiếu.

2. Data collection
   - Tìm ra các nguồn thông tin và mình cần, tuỳ thuộc vào tính chất của bài toán.
     - Kaggle, UCI Machine Learning Repository, government databases
     - Các APIs
     - Web Scraping: Trích xuất thông tin từ các website với các công cụ như Beautiful Soup Hoặc Scrapy
     - Surveys or interviews
     - Có thể sử dụng thông tin nội bộ nếu có.
   - Đánh giá chất lượng thông tin:
     - Accuracy: Có sai sót hay không đồng bộ?
     - Completeness: Có bao quát cho các điều kiện cần thiết?
     - Consistency: Thông tin có đồng bộ xuyên suốt các nguồn khác nhau?
     - Relevance: Thông tin có bao gồm nội dung mà mình xem xét?
     - Timeliness: Có mới/ hiện hành?

→ Trong suốt quá trình sử dụng data mình cần phân tích và điều chỉnh model của mình, có thể sẽ cần thêm data hoặc điều chỉnh data sẵn có.

3. Preprocessing

Được viết ở trên

4. Model Selection

   - Đánh giá xem bài toán đòi hỏi một mô hình như nào? (classification, regression, clustering,...) Cần phải hiểu về các feature, target, data size, data distribution và các pattern khác là gì?
   - Đánh giá Model Complexity and Interpretability. Xem xét độ phức tạp của mô hình và khả năng nắm bắt các mối quan hệ phức tạp trong dữ liệu. Các mô hình phức tạp hơn như mạng nơ-ron học sâu có thể mang lại độ chính xác dự đoán cao hơn nhưng có thể tốn kém về mặt tính toán. Các mô hình đơn giản như hồi quy tuyến tính dễ hiểu hơn so với các mô hình hộp đen phức tạp như mạng nơ-ron sâu...
   - Xem xét các thông số để đánh giá độ hiệu quả:
     - Classification: accuracy, precision, recall, F1-score, ROC-AUC, etc...
     - Regression: mean squared error (MSE), mean absolute error (MAE), R-squared, ...
     - Đánh giá khả năng tổng quát hoá của mô hình: cross-validation, train-test split, or time-based validation...
   - Bắt đầu với các mô hình cơ bản để tạo ra một tiêu chuẩn đánh giá. Sau đó có thể huấn luyện nhiều mô hình bằng các data set khác nhau và đánh giá chúng bằng các metric đã chọn.

   → Từ đó chọn ra model tốt nhất: Cân nhắc các sự khác biệt của các model với nhau, độ phức tạp, interpretability, computational resources, performance metrics. Sau đó đưa ra model tốt nhất để đảm bảo khả năng tổng quát hoá của chúng trên các dữ liệu mới.

5. Moden Training

   - Huấn luyện một model bao gồm việc áp dụng thuật toán đó vào training data để học các pattern và relationships. Quá trình này bao gồm việc chia

cắt data thành set training và validation.

- Chia cắt data: Phân tách data set thành các training, validation sets. Tỷ lệ chia đôi thường là 70-30 hoặc là 80-20 cho training/validation, đảm bảo là cái validation set tượng trưng cho phân bố data ngoài đời thật. (Một set để train, set còn lại là để kiểm tra)
- Lựa chọn thuật toán và sử dụng trên Python hoặc Scikit-Learn, VD:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
```

- Huấn luyện model bằng cách sử dụng .fit()
- Tối ưu hoá tham số. Hyperparameter là tham số mà mình chọn, bước này nói tới việc mình thay đổi các tham số đầu vào để tối ưu hoá mô hình. Có thể sử dụng các cách như grid search, random search, or Bayesian optimization.
- Đánh giá mô hình bằng cách sử dụng Validation set đã đặt ra ở trên. Tính toán các thông số chẳng hạn như accuracy, precision, recall, F1-score (for classification), or mean squared error như đã nói.
- Sau khi đã thoả mãn với kết quả của Validation Set thì có thể huấn luyện lại model bằng toàn bộ dataset để tối ưu hoá việc học trước khi đem ra sử dụng.

6. Model Deployment

- Serialize Model mình đã train vào một format phù hợp để sử dụng. Các format bao gồm pickle (Python), PMML (Predictive Model Markup Language), ONNX (Open Neural Network Exchange),  hoặc là một format tuỳ ý.
- Áp dụng vào môi trường thực bằng các framworks hoặc thư viện cụ thể (e.g., Flask for web APIs, TensorFlow Serving, or PyTorch serving for serving models).
- Phiên bản hoá hoặc ghi lại cách thay đổi để mình có thể rollback nếu cần thiết. Tạo một cái quy trình mà mình có thể update các model và huấn

luyện dựa trên các data mới và các thuật toán tốt hơn. Có thể sử dụng A/B testing để so sánh các model khác nhau trước khi đem vào sử dụng.